

Primitives of transformers

GLyC

October 28, 2025

Embedding with flags

Assume there is a transformer T with embedding size d . It's possible to add flags if we expand the embedding size. Since the dimensions of the embedding will be modified, it is needed to also change the dimensions of all the matrices and the token embedding. That can be done with zeros so it doesn't affect the behavior.

Since the positional embedding from T is just a mapping from $[n_{max}]$ to $\mathbb{R}^d$ it is the perfect place for marking the i th vector. It can be done defining the positional embedding of the transformer with flags as:

$$(\theta'_{TE}(n))_j = \begin{cases} (\theta_{TE}(n))_j & \text{if } j \neq d+1 \\ 1 & \text{if } j = d+1 \text{ and } n = i \\ 0 & \text{if } j = d+1 \text{ and } n \neq i \end{cases}$$

Note that is possible to flag multiple vectors in the same coordinate or to flag with a different value than 1. Also the flagging can be done in any coordinate, not only in the last one.

Add one specific vector to the i th

The i is fixed from the construction of the transformer, it's assumed that is flagged with a 1 in the last coordinate. Therefore all vectors will have 0 in their last coordinate with the exception of the i th which will have a 1.

Lets call v the vector that we want to add. We will do it with one layer of FF. We will construct a layer such that $FF(h_j) = \mathbb{I}(j = i) * v$.

$$W_1 = id \quad W_2 = \begin{pmatrix} 0 & \dots & 0 & v_1 \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \dots & 0 & v_d \end{pmatrix} \quad b_1 = b_2 = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix}$$

If we were to choose the k th coordinate for the flag instead of the last one, then we would have to put v in the k th column.

Make the i th vector zero

With just adding a fixed vector we can transform any vector to a constant one. For that we will first make it zero and then add the one we want. Lets see the first part.

For all x it happens that $(x + \infty) + \infty = \infty$. Since $x \geq -\infty$, it happens that $x + \infty \geq 0$, then $(x + \infty) + \infty \geq \infty$, so $(x + \infty) + \infty = \infty$. Therefore if we want to get zero, it's enough to subtract ∞ .

$$h_i \leftarrow \left(\left(h_i + \begin{pmatrix} \infty \\ \vdots \\ \infty \end{pmatrix} \right) + \begin{pmatrix} \infty \\ \vdots \\ \infty \end{pmatrix} \right) + \begin{pmatrix} -\infty \\ \vdots \\ -\infty \end{pmatrix}$$

ATTN y FF controlling which coordinates are affected

Later we will see that we want to go through layers of attention and feed forward ignoring some coordinates. That means without taking them into consideration and without affecting them in the output.

For example, if we want to ignore the i th coordinate through a layer of attention, it's enough to put zeros in the i th column and row of W_Q, W_K, W_V, W_O . This can be done with as many coordinates as desired. Note that because the i th row of W_O is zero, the output vector of the ATTN layer will have a zero in the i th coordinate. This is the point that makes it possible to not just ignore the value of the i th coordinate but to not affect it. This happens because the result of the attention is added to the original vector.

The same applies in the feed forward layer, putting zeros in the i th column and row of the matrix and vectors of the FF makes it ignore the value of the i th coordinate. Also in the result of the FF will appear a zero in the i th coordinate.

Linear transformations

We define two ways of applying linear transformations to an specific vector. Both have the effect of corrupting every vector except the one with the result. The first one saves the answer in a different vector to the right of the one receiving the transformation. Note that this requires adding a extra character at the end (so we get an extra vector) if we want to apply a transformation to the last vector, which we will need for the normalizations. This is uncomfortable for doing CoT. That's why we also introduce the second way of applying linear transformation which saves the answer in the same vector, but requires to double the embedding size.

Linear transformation storing to the right

In this section we will see how to apply a linear transformation to one of the vectors and save the output in another more to the right, in other words: $h_j^{l+c} = Mh_i^l$ for some $i < j$ previously chosen.

Note that after this procedure all the vector will have their values corrupted with the exception of h_j .

First we will make $h_j = 0$ at the coordinates we are interested (this can be achieved with the previous sections). Then with an ATTN layer we will add Mh_i .

Using the previous sections we can assume that every vector's last two coordinates are $(0, 1)$, except h_i 's which are $(1, 0)$. These numbers would have been set by the encoding and preserved through the previous layers.

Assuming this we will construct W_K and W_Q such that:

$$\langle q_j, k_{i'} \rangle = \begin{cases} -\infty & \text{if } i' \neq i \\ 0 & \text{if } i' = i \end{cases}$$

Note that the W_Q and W_K that we are looking for are the following:

$$W_Q = \begin{pmatrix} 0 & \dots & 0 & 0 \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \dots & 0 & 0 \\ 0 & \dots & 0 & -\infty \end{pmatrix} \quad W_K = \begin{pmatrix} 0 & \dots & 0 & 0 & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & \dots & 0 & 0 & 0 \\ 0 & \dots & 0 & 0 & 1 \end{pmatrix}$$

since

$$q_j = W_Q h_j = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ -\infty \end{pmatrix} \quad k_{i'} = W_K h_{i'} = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ (h_{i'})_d \end{pmatrix}$$

so $\langle q_j, k_{i'} \rangle = -\infty * (h_{i'})_d$

Taking $W_O = M$ y $W_V = id$ we get what we were looking for.

This works because $e^{-\infty} = 0$ and $\text{softmax}(-\infty, \dots, -\infty, 0) = (0, \dots, 0, 1)$, which makes $(s_j)_{i'} = \mathbb{I}(i' = i)$. In summary:

$$\begin{aligned} W_O \sum_{i'=0}^n (s_j)_{i'} v_{i'} &= W_O \sum_{i'=0}^j (s_j)_{i'} v_{i'} = W_O \sum_{i'=0}^j (s_j)_{i'} h_{i'} = \\ &= W_O \sum_{i'=0}^j \mathbb{I}(i' = i) h_{i'} = W_O h_i \end{aligned}$$

Linear transformation doubling the embedding size

As it was said before, if we want to store the result of the linear transformation in the same vector (because of the error given by the finite representation)¹ we will have to double the embedding size.

The linear transformation will be realized through two layers of ATTN. The first layer will store the answer of the linear transformation in the second half of the embedding and overwrite the first one with zeros. Then, the second layer will swap the two parts so the answer is finally stored in the first part of the embedding. It will also overwrite the second half with zeros so it can be reused.

First, it's needed to extend all previous matrices and embeddings from dimension d to dimension $2 * d + 1$. This has to be done with zeros in everyplace with the exception of the positional embedding and the last coordinate of the vector we want to multiply. The plus 1 came from the flag and the times 2 for having space to save the result.

As we assumed before, the vector we will want to multiply by a matrix will be flagged with a 1 in the last coordinate and all the other will have a 0. As seen before the matrices for W_Q and W_K will be such that h_i only attends to its self:

$$W_Q = W_K = \begin{pmatrix} 0 & \dots & 0 & 0 \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \dots & 0 & 0 \\ 0 & \dots & 0 & \infty \end{pmatrix}$$

Then,

$$(W_Q h_{i'})_j = (W_K h_{i'})_j = \begin{cases} 0 & \text{if } j \neq 2 * d + 1 \\ 0 & \text{if } j = 2 * d + 1 \wedge i' \neq i \\ \infty & \text{if } j = 2 * d + 1 \wedge i' = i \end{cases}$$

W_V will be the identity and W_O will be:

$$W_O = \begin{pmatrix} -id^{\mathbb{R}^{d \times d}} & 0^{\mathbb{R}^{d \times d}} & 0^{\mathbb{R}^d} \\ M & 0^{\mathbb{R}^{d \times d}} & 0^{\mathbb{R}^d} \\ 0^{\mathbb{R}^{1 \times d}} & 0^{\mathbb{R}^{1 \times d}} & 0^{\mathbb{R}} \end{pmatrix}$$

Where M is the matrix of the linear transformation wanted to apply.

Note that in the ATTN vector, all the values will be 0 except for $\langle q_i, k_i \rangle$ which will be 1. Let's call h_i^1 to the first half of h_i . Then, the output of the ATTN layer of the i th vector will be: $(-h_i^1, Mh_i^1, 0)$ which added to h_i gives us $(0^d, Mh_i^1, 1)$.

¹maybe expand this?

In the second layer we only need to swap the halves of h_i . For that we will use the same query key and value matrices but the W_O will be:

$$W_O = \begin{pmatrix} 0 & id^{\mathbb{R}^{d \times d}} & 0^{\mathbb{R}^d} \\ -id^{\mathbb{R}^{d \times d}} & 0^{\mathbb{R}^{d \times d}} & 0^{\mathbb{R}^d} \\ 0^{\mathbb{R}^{1 \times d}} & 0^{\mathbb{R}^{1 \times d}} & 0^{\mathbb{R}} \end{pmatrix}$$

Then the output in the i th vector of this layer of ATTN will be $(Mh_i, -Mh_i, 0)$ so after adding it with the previous one we get $(Mh_i, 0^{d+1})$. Exactly what we were looking for.

Maximum of two numbers

A useful mechanic is being able to do this transformation:

$$\begin{pmatrix} a \\ b \end{pmatrix} \rightarrow \begin{cases} \begin{pmatrix} a \\ 0 \end{pmatrix} & \text{if } a > b \\ \begin{pmatrix} 0 \\ b \end{pmatrix} & \text{if } b > a \end{cases}$$

We will show how to do it with two coordinates and then it's easily extended to more.

$$\begin{aligned} \begin{pmatrix} a \\ b \\ \vdots \end{pmatrix} &\xrightarrow{\text{FF layer}} \begin{pmatrix} a \\ b \\ \text{relu}(a-b) \\ \text{relu}(b-a) \\ \vdots \end{pmatrix} \xrightarrow[\text{the coordinates with the relu}]{\text{Multiply twice by } \infty} \begin{pmatrix} a \\ b \\ \infty \times \mathbb{I}(a > b) \\ \infty \times \mathbb{I}(b > a) \\ \vdots \end{pmatrix} \rightarrow \\ &\xrightarrow[\text{the comparisons}]{\text{Divide by } \infty \text{ the coordinates with}} \begin{pmatrix} a \\ b \\ \mathbb{I}(a > b) \\ \mathbb{I}(b > a) \\ \vdots \end{pmatrix} \xrightarrow[\text{operation}]{\text{Special}} \begin{pmatrix} a \times \mathbb{I}(a > b) \\ b \times \mathbb{I}(b > a) \\ \vdots \end{pmatrix} = \begin{cases} \begin{pmatrix} a \\ 0 \\ \vdots \end{pmatrix} & \text{if } a > b \\ \begin{pmatrix} 0 \\ b \\ \vdots \end{pmatrix} & \text{if } b > a \end{cases} \end{aligned}$$

If we want to apply this operation to more than two coordinates it can be done applying it in pairs and then doing maximum of the addition of the pairs. The expressions will be slightly more complex but the idea remains the same (because it's the same process but it needs to affect more coordinates).

$$\begin{aligned}
\begin{pmatrix} a \\ b \\ c \\ d \\ \vdots \end{pmatrix} &\rightarrow \begin{pmatrix} a \times \mathbb{I}(a > b) \\ b \times \mathbb{I}(b > a) \\ c \times \mathbb{I}(c > d) \\ d \times \mathbb{I}(d > c) \\ \vdots \end{pmatrix} = \begin{pmatrix} \alpha \\ \beta \\ \gamma \\ \delta \\ \vdots \end{pmatrix} \rightarrow \begin{pmatrix} \alpha \\ \beta \\ \gamma \\ \delta \\ \text{relu}((\alpha + \beta) - (\gamma + \delta)) \\ \text{relu}((\gamma + \delta) - (\alpha + \beta)) \\ \vdots \end{pmatrix} \rightarrow \\
&\rightarrow \dots \rightarrow \begin{pmatrix} \alpha \times \mathbb{I}((\alpha + \beta) - (\gamma + \delta)) \\ \beta \times \mathbb{I}((\alpha + \beta) - (\gamma + \delta)) \\ \gamma \times \mathbb{I}((\gamma + \delta) - (\alpha + \beta)) \\ \delta \times \mathbb{I}((\gamma + \delta) - (\alpha + \beta)) \\ \vdots \end{pmatrix}
\end{aligned}$$

Note that either α or β it's zero and the other one it's the maximum between a or b (if the maximum is a then β will be zero and respectively if the maximum is b , α will be zero). The same happens with c, γ, d, δ ,

Clearly this process can be extend to more coordinates.

Special operation

The operation we are looking for to do its not a proper linear transformation but with the infinity trick it can be done.

$$\begin{pmatrix} a \\ b \\ \mathbb{I}_a \\ \mathbb{I}_b \\ \dots \end{pmatrix} \xrightarrow[\text{operation}]{\text{Special}} \begin{pmatrix} a \times \mathbb{I}_a \\ b \times \mathbb{I}_b \\ \dots \end{pmatrix}$$

It can be performed with 3 layers of FF. The first two will be:

$$W_1 = id \quad W_2 = \begin{pmatrix} 0 & \dots & 0 & 0 & \infty \\ \vdots & \ddots & \vdots & \infty & 0 \\ \vdots & \ddots & \vdots & 0 & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & \dots & 0 & 0 & 0 \end{pmatrix} \quad b_1 = b_2 = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix}$$

since the output of this FF when inputted with $(a, b, \mathbb{I}_a, \mathbb{I}_b, \dots)$ will be:

$$FF \left(\begin{pmatrix} a \\ b \\ \mathbb{I}_a \\ \mathbb{I}_b \\ \dots \end{pmatrix} \right) = \begin{pmatrix} \infty \times \mathbb{I}_b \\ \infty \times \mathbb{I}_a \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

After two iterations of this the FF the smaller one between the first and second coordinate will be infinity and the other one will have the value unmodified.

Now it's enough to subtract infinity from the coordinate that had originally the smaller value. That can be done with almost the same FF:

$$W_1 = id \quad W_2 = \begin{pmatrix} 0 & \dots & 0 & 0 & -\infty \\ \vdots & \ddots & \vdots & -\infty & 0 \\ \vdots & \ddots & \vdots & 0 & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & \dots & 0 & 0 & 0 \end{pmatrix} \quad b_1 = b_2 = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix}$$

In summary we did:

$$\begin{pmatrix} a \\ b \\ \mathbb{I}_a \\ \mathbb{I}_b \\ \dots \end{pmatrix} \xrightarrow[\text{operation}]{\text{Special}} \begin{pmatrix} a + \infty \times \mathbb{I}_a + \infty \times \mathbb{I}_a - \infty \times \mathbb{I}_a \\ b + \infty \times \mathbb{I}_b + \infty \times \mathbb{I}_b - \infty \times \mathbb{I}_b \\ \dots \end{pmatrix}$$

Which thanks to the finite representation it's exactly what we want.